

Claude Code

GHID COMPLET · TERMINAL

Manual de la zero la avansat, structurat după utilitate: începe cu ce folosești zilnic și urcă spre funcții avansate dar utile. Construit din documentația oficială v2.1.161 + analiză externă verificată.

Model implicit: **Opus 4.8** (efort **high**) · Caută în PDF cu **Cmd+F** · Lista mereu actuală de comenzi: **/help**

CUPRINS — PE PĂRȚI, DE LA ZILNIC LA AVANSAT

I · ZILNIC

1. Esențialul (start rapid)
 2. Prefixe în prompt
 3. Taste & navigare
 4. Moduri de permisiuni
 5. Vezi ce s-a modificat
 6. Sesiune & context
- ### II · LUCRUL CU CODUL
7. Cum & când începi o sesiune
 8. Top 10 use cases
 9. Cod, git & calitate
 10. Prompting pentru calitate
 11. Greșeli de evitat

III · CONTROL

12. Model, efort & thinking
 13. Cost, viteză & limite
- ### IV · PERSONALIZARE
14. Memorie & CLAUDE.md
 15. Comenzi config & info
 16. settings.json & env
 17. Hooks (automatizări)

V · AVANSAT DAR UTIL

18. Lucru în paralel
 19. Funcții avansate
 20. Comenzi de pornire (CLI)
 21. Top tools & setup
 22. Monitorizare & productivitate
- ### VI · MEDIU & REFERINȚĂ
23. Top 3 terminale
 24. Ghostty — setup
 25. Depanare rapidă
 26. Glosar
 27. Flux de lucru (rezumat)

PARTEA I Zilnic — ce folosești în fiecare zi

start aici

1 Esențialul — învață astea primele

Shift+Tab	Comută modul de permisiuni: Normal → Accept edits → Plan . Cea mai folosită tastă.
/ + nume	Slash commands. Scrie doar / ca să vezi tot. /help = lista completă.
@ + cale	Atașezi fișier/folder în context. Tab = autocomplete pe căi.
! + comandă	Rulezi shell direct; output-ul intră în conversație.
# + text	Salvezi un fapt în memorie — persistă între sesiuni.
Esc	Oprești instant ce face Claude. Esc Esc = editezi un mesaj anterior / rewind.
/clear	Cureți contextul când treci la alt task — evită amestecul.

2 Prefixe în prompt

<code>/ slash</code>	Comenzi & skill-uri (built-in, ale tale, din pluginuri/MCP). Filtrezi tastând litere.
<code>@ fișier</code>	<code>@src/auth.ts</code> = include conținutul · <code>@folder/</code> = listează · <code>@server: resursă</code> = resursă MCP.
<code>! shell</code>	Comandă fără ca Claude s-o interpreteze. Background cu <code>Ctrl+B</code> . Ieși cu <code>Esc</code> .
<code># memorie</code>	Salvezi rapid un fapt în memoria persistentă (preferințe, convenții).
<code>/btw ...</code>	Întrebare laterală rapidă despre conversație, fără s-o adaugi în istoric (cost mic).
Imagini	Lipești (<code>Ctrl+V</code>) sau tragi o imagine în prompt → Claude o analizează (screenshot bug, mockup).

3 Taste & navigare

Control general

<code>Shift+Tab</code>	Comută mod permisiuni
<code>Esc</code>	Întrerupe acțiunea
<code>Esc Esc</code>	Editează mesaj / rewind
<code>Ctrl+C</code>	Anulează input (x2 = ieși)
<code>Ctrl+D</code>	Ieși din sesiune
<code>Ctrl+L</code>	Redesenează ecranul
<code>Ctrl+R</code>	Caută în istoric
<code>Ctrl+O</code>	Arată/ascunzi raționamentul
<code>Ctrl+B</code>	Trimite task în fundal
<code>↑ / ↓</code>	Istoric mesaje

macOS — Option / editare

<code>Option+T</code>	Thinking on/off
<code>Option+P</code>	Schimbă modelul
<code>Option+O</code>	Fast mode on/off
<code>Ctrl+G</code>	Editează prompt/plan în \$EDITOR
<code>Ctrl+A / E</code>	Început / sfârșit de linie
<code>Ctrl+W</code>	Șterge cuvântul anterior

Linie nouă (multi-linie)

<code>\ + Enter</code>	Merge oriunde
<code>Ctrl+J</code>	Alternativă universală
<code>Shift+Enter</code>	Nativ pe Ghostty/iTerm2/Terminal; în VSCode rulează <code>/terminal-setup</code>

4 Moduri de permisiuni

comută cu Shift+Tab

Normal (default)	Cere aprobare la fiecare editare/comandă. Cel mai sigur pentru început.
Accept edits	Aplică editări + comenzi fs sigure în working dir fără să întrebe. Pentru iterare rapidă.
Plan mode	Citește & explorează, scrie un plan, dar nu modifică nimic . La final alegi cum execută.
Auto <code>PREVIEW</code>	Rulează fără prompturi; un clasificator blochează acțiunile periculoase (deploy, force push, ștergeri).
Bypass <code>RISC</code>	Fără verificări. Doar în containere/VM izolate (<code>--dangerously-skip-permissions</code>).

Control fin: `/permissions` = allow/deny pe comenzi specifice (ex. `Bash(npm run test:*)`) ca să nu te mai întrebe la cele repetitive.

5 Vezi ce s-a modificat / adăugat

Diff inline

Automat în chat la fiecare editare — verde = adăugat, roșu = șters.

`! git status`

Fișiere: **M** modificat · **??** nou · **A** adăugat.

`! git diff`

Diff complet. **--stat** = sumar pe fișiere.

`/diff`

Vizualizator interactiv (git vs. turn-uri, navighezi fișierele).

Integrare IDE

Rulat din terminalul VSCode → diff-urile apar în panoul nativ.

6 Sesiune & gestionarea contextului

`/clear`

Conversație nouă, păstrează memoria proiectului. **Folosește între task-uri.**

`/compact [instr]`

Comprimă conversația păstrând esențialul; poți spune pe ce să se concentreze.

`/context`

Cât din fereastra de context e folosită + sugestii.

`/rewind`

Derulează conversația/codul la un checkpoint anterior (alias **/undo**).

`/resume` · `/branch`

Reia o sesiune din listă · creează o ramură din punctul curent.

`/export` · `/copy`

Exportă conversația · copiezi ultimul răspuns.

`claude -c`

Din shell: reia ultima sesiune (alias **--continue**).

7 Cum & când începi o sesiune

Cum începi o sesiune nouă (recomandat)

Prima dată în proiect `/init` → generează CLAUDE.md din codebase, apoi îl ajustezi și-l comiți.

Task complex Plan mode (`Shift+Tab`): explore → plan → aprobi → implementează → commit.

Feature mare Pune-l să te **intervieveze** întâi (spec în SPEC.md), apoi implementezi în sesiune **curată**.

Fii specific Menționează fișiere cu `@`, constrângeri, exemple. Prompt precis = mai puține corecții.

Dă-i o țintă verificabilă Teste/build/lint/screenshot pe care le rulează singur → se auto-corectează fără tine.

Când începi sesiune nouă / cureți contextul

`/clear` Între task-uri **fără legătură**. Contextul e cea mai importantă resursă.

După 2 corecții eșuate `/clear` + prompt mai bun. O sesiune curată bate aproape mereu una lungă plină de ratări.

`/compact` [focus] Sesiune lungă dar coerentă: comprimă păstrând esențialul.

Sub-agenți „use subagents to investigate X” — context separat, conversația principală rămâne curată.

8 Top 10 use cases documentate

1. Înțelegi un codebase nou Întrebări directe / Explore: „cum funcționează auth în `src/auth/`”.

2. Repari un bug Lipești eroarea/stack trace-ul; găsește cauza, repară + teste.

3. Implementezi un feature Interviu → spec (SPEC.md) → implementezi în sesiune curată.

4. Teste & TDD Generează suite de teste, rulează, iterează până trec.

5. Code review `/code-review` pe diff/PR, în context curat.

6. Refactor & migrări Fan-out pe fișiere; rulări paralele cu `--allowedTools` limitat.

7. Git & PR-uri Commit-uri descriptive, branch-uri, deschide PR-uri.

8. Lint / format / dependențe Repară lint la scară, update dependențe, rezolvă conflicte.

9. Verificare vizuală UI Lipești mockup; implementează, face screenshot, compară, iterează.

10. Documentație & automatizări Release notes, API docs, comentarii; task-uri non-cod din CLI.

9 Cod, git & calitate

`/code-review` Review pe diff: bug-uri + curățenie. Niveluri `low-max`; `ultra` = cloud multi-agent.
`--fix` aplică, `--comment` postează pe PR.

`/simplify` Doar curățenie (reuse, simplificare, eficiență) — nu caută bug-uri.

`/security-review` Caută vulnerabilități (injection, auth, expunere date).

`/run` · `/verify` Pornește app-ul și confirmă vizual că schimbarea funcționează.

`/init` Generează un `CLAUDE.md` documentând codebase-ul.

⚠️ **Reguli de aur înainte de commit** Citește diff-ul (`! git diff`), testele **trebuie să treacă**, și cere dovada (output real), nu doar „s-a făcut”.

10 Prompting pentru rezultate de calitate

✓ Fă așa

- ✓ Specific: **ce, unde** (fișier/funcție), **de ce**.
- ✓ Dă context: erori complete, log-uri, așteptat vs. real.
- ✓ Cere **planul întâi** la task-uri mari (Plan mode).
- ✓ Spune **cum verifici**: „rulează testele X”.
- ✓ Atașează fișiere cu **@** în loc să-l pui să caute.
- ✓ Lucrează **iterativ**: plan → aprobi → bucăți → verifici.

✗ Evită

- ✗ „Repară bug-ul” fără simptom sau locație.
- ✗ Cereri vagi: „fă-l mai bun” fără criteriu.
- ✗ Task-uri fără legătură în aceeași sesiune (dă **/clear**).
- ✗ Să presupui că-ți ghicește convențiile — pune-le în **CLAUDE.md**.
- ✗ Să accepți „ar trebui să meargă” — cere dovada.

💡 **Pârghii de calitate** La probleme grele: urcă **/effort** la **xhigh / max** sau pune **ultrathink** în prompt. La final: **/code-review** + testele.

11 Greșeli comune de evitat

din „best practices” oficial

✗ CLAUDE.md umflat	Fișierele prea mari fac Claude să ignore instrucțiunile reale . Ține-l sub ~200 linii.
✗ Corectezi la nesfârșit	După 2 corecții eșuate , dă /clear și rescrie. Sesiunea curată bate una lungă.
✗ Context lung & irelevant	Performanța scade când contextul se umple. /clear + sub-agenti.
✗ Tu ești bucla de verificare	Fără țintă verificabilă, fiecare greșeală te așteaptă pe tine. Dă-i un check.
✗ Nu-l oprești la timp	Esc imediat ce o ia razna — contextul se păstrează.
✗ Prompturi vagi	„Fă-l mai bun” cere multă pilotare. Fii specific.
✗ Commit fără diff	Tu rămâi responsabil. ! git diff + dovada testelor.

12 Model, efort & thinking

Nivel de efort — echivalentul lui high/medium/low

`/effort`

Slider. Pe **Opus 4.8**: `low` · `medium` · `high` ← `default` · `xhigh` · `max`. Direct: `/effort max`.

Alte căi

În `/model` cu `←` `→` · la pornire `--effort high` · permanent `"effortLevel"` în settings.

Reset

La schimbarea modelului, efortul revine la default-ul acelui model.

Modele & thinking

`/model`

Opus = task-uri grele · **Sonnet** = munca zilnică (consum mic) · **Haiku** = rapid/simplu.

`/fast`

Output mai rapid pe Opus 4.8 — rămâne tot Opus.

`Option+T`

Thinking on/off pentru sesiune. `/config` = default global.

`ultrathink`

Pus în prompt = gândire profundă pt. acel mesaj. **Doar acest cuvânt** e recunoscut.

`Ctrl+0`

Arată/ascunde raționamentul (ascuns by default).

13 Optimizare cost, viteză & limite

Alege modelul potrivit

Sonnet zilnic (consum mic) · **Opus** doar la task-uri grele · **Haiku** simple. `/model` comută fără a pierde contextul.

Reglează efortul

`/effort low` / `medium` la simple economisește; urci doar când trebuie.

`/fast`

Viteză pe Opus fără să cobori la model mai mic.

`/clear` des

Context mic = răspunsuri mai bune **și** consum mai mic. Câștig dublu.

Monitorizezi

`/usage` = consum + reset (5h & săptămânal) · `/cost` = sesiune · `/context` = ocupare.

14 Memorie & instrucțiuni permanente (CLAUDE.md)

Cea mai bună investiție pentru calitate: pune-ți convențiile o dată, le respectă mereu.

<code>CLAUDE.md</code>	Fișier în repo, citit la pornire: convenții de cod, comenzi build/test, ce vrei să știe constant.
<code>~/.claude/CLAUDE.md</code>	Preferințe globale, în toate proiectele tale.
<code>CLAUDE.local.md</code>	Note personale per-proiect (în <code>.gitignore</code>).
<code>@cale</code> în <code>CLAUDE.md</code>	Importă alt fișier în instrucțiuni (până la 4 nivele).
<code>.claude/rules/</code>	Reguli pe tipuri de fișiere (cu <code>paths:</code> în frontmatter).
<code>/init</code> · <code>/memory</code> · <code>#</code>	Generezi CLAUDE.md · editezi memoria · salvezi rapid un fapt.

15 Comenzi config & info

<code>/help</code>	Toate comenzile	<code>/status</code> · <code>/doctor</code>	Info · diagnostic
<code>/config</code>	Setări (model, temă, editor)	<code>/permissions</code>	Reguli allow/deny
<code>/model</code> · <code>/effort</code>	Model · efort	<code>/hooks</code> · <code>/mcp</code>	Automatizări · servere MCP
<code>/fast</code>	Fast mode	<code>/agents</code>	Sub-agenti
<code>/usage</code> · <code>/cost</code>	Consum · cost	<code>/theme</code> · <code>/statusline</code>	Temă · bară status

16 Config: settings.json & variabile de mediu

Precedență: `managed` → `CLI` → `.local.json` → `project` → `user` (`~/.claude/settings.json`).

<code>model</code> · <code>effortLevel</code>	Model & efort implicit.
<code>permissions</code>	<code>allow</code> / <code>ask</code> / <code>deny</code> + <code>defaultMode</code> + <code>additionalDirectories</code> .
<code>env</code>	Variabile injectate în comenzi.
<code>hooks</code> · <code>statusLine</code>	Automatizări · bară de status custom.
<code>includeCoAuthoredBy</code>	Trailer co-authored-by la commit-uri.
<code>editorMode</code>	<code>vim</code> sau <code>normal</code> .

Env utile: `ANTHROPIC_MODEL` · `CLAUDE_CODE_EFFORT_LEVEL` · `BASH_DEFAULT_TIMEOUT_MS` · `MAX_THINKING_TOKENS=0` (oprește gândirea) · `DISABLE_TELEMETRY=1` .

17 Hooks — automatizări (exemple)

harness-ul le execută, nu Claude

Comenzi care rulează automat la evenimente. Le pui în `settings.json`; le vezi cu `/hooks` . Pentru reguli „de fiecare dată când...” folosește `hooks`, nu memoria.

<code>PostToolUse</code>	După o editare (ex. format/lint automat).
<code>PreToolUse</code>	Înainte de o comandă — poate bloca (ex. <code>rm</code> periculos).
<code>SessionStart</code>	La pornire (ex. injectezi context/reguli).
<code>Stop</code> · <code>Notification</code>	Când termină · când are nevoie de tine (ex. sunet/notificare).

Exemplu: format automat după fiecare editare (în `~/.claude/settings.json`):

```
// rulează prettier pe fișierul editat
"hooks": {
  "PostToolUse": [{
```

```
"matcher": "Edit|Write",
"hooks": [{
  "type": "command",
  "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
}]
}
```

Tip: skill-ul `/update-config` te ajută să configurezi hooks corect, fără să scrii JSON-ul de mână.

18 Lucru în paralel & multiple sesiuni

„windows” ca în VSCode

1. Taburi / ferestre
Mai multe taburi (Ghostty: `Cmd+T`), split (`Cmd+D`), `claude` în fiecare. **Atenție** dacă lucrează pe aceleași fișiere.
2. Git worktrees **RECOMANDAT**
`claude --worktree feature-x` = sesiune pe worktree izolat (alt branch, fișiere separate). Două = paralel care **nu se ating**.
3. Sub-agenți
Paralel **în interiorul** unei sesiuni: „use subagents to investigate X”. Nu-ți umplu conversația.
4. Task-uri în fundal
`Ctrl+B` trimite o comandă lungă în fundal; continui să lucrezi. `/tasks` le listează.
5. Agent Teams **EXPERIMENTAL**
Sesiuni coordonate cu listă comună & mesagerie, conduse de un „lead”.

19 Funcții avansate

Sub-agenți

- › Asistenți izolați cu prompt & unelte proprii — nu umplu contextul principal.
- › Built-in: **Explore, Plan, general-purpose**.
- › Proprii în `.claude/agents/*.md`; gestionezi cu `/agents`.

Plan mode & rewind

- › Plan mode: explorează → plan → alegi execuția. `Ctrl+G` editezi planul.
- › `/rewind` restaurează fișierele la un checkpoint (nu dă revert la commit-uri git).

MCP (unelte externe)

- › Conectezi servicii (GitHub, DB, browser...). Tu ai `mobile-mcp`.
- › `claude mcp add` sau `.mcp.json`; gestionezi cu `/mcp`.

Headless / scripting

- › `claude -p "task"` = rulare non-interactivă (scripturi, CI).
- › `--output-format json` structurat; `--allowedTools` pre-aprobă.

IDE: rulat din terminalul VSCode/JetBrains, `/ide` aduce diff-urile & selecția în panoul nativ.

20 Comenzi de pornire (din shell)

Pornire sesiune

<code>claude</code>	Interactiv
<code>claude -c</code>	Reia ultima sesiune
<code>claude --resume</code>	Alegi din listă
<code>claude -p "..."</code>	Non-interactiv (script)
<code>--model opus</code>	Modelul
<code>--effort high</code>	Efortul
<code>--worktree nume</code>	Worktree nou izolat

Subcomenzi `claude ...`

<code>claude update</code>	Actualizează
<code>claude doctor</code>	Verifică instalarea
<code>claude mcp ...</code>	add/list/remove MCP
<code>claude plugin ...</code>	install/list pluginuri
<code>claude --help</code>	Toate opțiunile
<code>claude --version</code>	Versiunea

21 Top tools & setup recomandat

stars verificate iun. 2026

★ **Fundația care dă cel mai mult (înainte de orice tool) CLAUDE.md scurt + bucla `explore→plan→code→commit` + un `check verificabil`. Asta aduce cel mai mare salt, nu o unealtă.**

MCP Servers (oficial) ★ ~86.7K Servere de referință (fs/git/fetch). **Pentru Claude Code CLI sunt redundante** (le face deja nativ) — nu îți trebuie.

Context7	★ ~56.6K	Cel mai util pt. tine: docs up-to-date pe versiunea exactă (Firebase/TS/Android). <code>npx ctx7 setup --claude</code> → scrii <code>use context7</code> .
wshobson/agents	★ ~36.3K	Cel mai mare toolkit comunitar (192 sub-agenți, 156 skills). Opțional. <code>/plugin</code> .
Playwright MCP	★ ~33.4K	Automatizare browser (Microsoft). <code>claude mcp add playwright npx @playwright/mcp@latest</code>
GitHub MCP	OFICIAL	Repo/issues/PR/Actions.

Setup pt. WifiPass: `CLAUDE.md` (`/init`) + **Context7** + **GitHub MCP** + `mobile-mcp` (ai deja). MCP fs/git oficiale = nu necesare.

22

Monitorizare consum & productivitate

Max plan · stars verificate iun. 2026

★ **Vezi consumul mereu, fără să rulezi o comandă**Pune o **status line** (bară persistentă jos). Din **v2.1.132+** JSON-ul ei conține câmpurile oficiale `rate_limits.five_hour` și `rate_limits.seven_day` → vezi **fereastra de 5h + limita săptămânală** cu countdown la reset. Apar după primul răspuns API din sesiune.

Monitorizare consum (abonament)

Status line	NATIV	Funcție built-in, dar goală by default . Activează cu <code>/statusline</code> sau pune un script în <code>settings.json</code> . <code>/usage</code> = aceleași limite, manual.
ccstatusline	★ ~10.2K	Recomandat: statusline gata-făcut cu bare de progres pe 5h/săptămânal, config vizual (TUI). <code>npm i -g ccstatusline</code> → în <code>settings.json</code> : <code>"command":"ccstatusline"</code> .
ccusage	★ ~15.5K	Rapoarte de consum: zi / lună / sesiune / block de 5h. <code>npx ccusage@latest</code> (<code>daily</code> , <code>blocks</code>).
Usage-Monitor	★ ~8.1K	Monitor live în terminal cu predicții ML & avertizări (Maciek-roboblog).

Pe Max 20x dolarii din ccusage = echivalent API pay-as-you-go (irelevanți, plătești fix). Uită-te la % din 5h / săptămânal, nu la \$.

Top tools de productivitate

claude-code-templates	★ ~27.7K	Cel mai bun start: „app store” CLI — instalezi agenți/comenzi/hooks/MCP dintr-un catalog + dashboard analytics. <code>npx claude-code-templates@latest</code> .
claude-code-router	★ ~34.7K	Plasă pe limita săptămânală: rutează task-uri către alte modele (local/ieftin) când atingi capul. <code>npm i -g @musistudio/claude-code-router</code> . Ține Opus pe ce contează.
BMAD-METHOD	★ ~48.5K	Framework de workflow structurat (PRD→arhitectură→story→dev). Pentru features mari ; e heavy pt. fix-uri mici.
SuperClaude	★ ~23.2K	Alternativă mai ușoară la BMAD: comenzi + persona + eficiență de token, fără ceremonia agile.
claude-squad	★ ~7.7K	Rulezi mai mulți agenți în paralel (tmux) — ataci 2-3 task-uri simultan.

Directorul de referință: `hesreallyhim/awesome-claude-code` ★ ~45.6K — îl deschizi când cauți ceva nou. **Regula de aur:** instalează punctual, nu tot — fiecare agent/MCP adaugă context.

23 Top 3 terminale pentru Claude Code

analiză externă · iun. 2026

Criterii: Shift+Enter nativ, notificări desktop native, lipire imagini, rendering rapid.

 Ghostty  ~55.8K mac · Linux	Cea mai recomandată în 2026. Shift+Enter nativ, notificări native fără setup , rendering GPU cel mai rapid, zero-config. Lipire imagini cu Ctrl+V ; nu merge peste SSH.
 iTerm2 mac	Default-ul macOS, cel mai documentat. Shift+Enter + notificări native. Cmd+V la imagini. Setup: Option→„Esc+”, /terminal-setup + restart. Mai lent.
 WezTerm  ~26.4K mac·Linux·Win	Singurul cross-platform. Shift+Enter nativ, multiplexer încorporat (înlocuiește tmux), toate protocoalele de imagini.
Mențiune: Kitty  ~33.3K	Aproape la egalitate; arguabil #1 pe Linux.
 Evită: Warp	Modelul „block-based” se ceartă cu output-ul streaming; fără notificări native.

24 Ghostty — setup pentru Claude Code

Instalare (mac)	brew install --cask ghostty
Config	~/ .config/ghostty/config
Reload	Cmd+Shift+,
Teme (preview)	ghostty +list-themes
Tab / split	Cmd+T · Cmd+D

Config recomandat (copy-paste). Cea mai populară temă = Catppuccin, auto light/dark.

```
# ~/.config/ghostty/config — Catppuccin (auto dark/light)
theme = dark:Catppuccin Mocha,light:Catppuccin Latte
font-family = JetBrains Mono
font-size = 14
font-thicken = true
background-opacity = 0.96
background-blur = 20
window-padding-x = 14
window-padding-y = 14
window-padding-balance = true
macos-titlebar-style = tabs
cursor-style = block
mouse-hide-while-typing = true
copy-on-select = clipboard
```

 **Capcană la nume (Ghostty 1.2.0+)**Title Case cu spații; vechile kebab-case **nu mai merg**. **TokyoNight** = **fără spațiu**, dar **Catppuccin Mocha** & **Gruvbox Dark** au spațiu. Confirmă cu **ghostty +list-themes**. Populare: **Dracula** · **Nord** · **Gruvbox Dark** · **Rose Pine**.

25 Depanare rapidă

Ceva nu merge / e ciudat	/doctor (apasă f = auto-fix) · claude doctor din shell.
Claude pare „prost” azi	Probabil context aglomerat → /clear . Verifică modelul cu /model și efortul cu /effort .
Răspunsuri lente	/fast , sau coboară /effort , sau treci pe Sonnet.
A atins limita	/usage arată cât mai e până la reset (5h / săptămânal).

Mă întreabă prea des	<code>/permissions</code> → allow pe comenzile repetitive; sau <code>Shift+Tab</code> → Accept edits.
Shift+Enter nu merge	În VSCode/Cursor/Alacritty rulează <code>/terminal-setup</code> o dată.
Vreau să raportez un bug	<code>/bug</code> sau <code>/feedback</code> .
Am stricat ceva	<code>/rewind</code> (fișiere la checkpoint) sau <code>! git diff</code> + <code>git restore</code> .

26

Glosar — termeni cheie

Context window	„Memoria de lucru” a sesiunii: tot ce s-a discutat + fișiere citite. Plină = performanță mai slabă.
CLAUDE.md	Fișier de instrucțiuni citit la pornire — convențiile proiectului tău.
Sub-agent	Asistent izolat care face un sub-task în contextul lui și aduce doar concluzia.
MCP	Protocol prin care Claude se conectează la unelte externe (DB, GitHub, browser, mobil).
Hook	Comandă rulată automat de harness la un eveniment (ex. format după editare).
Worktree	Copie izolată a repo-ului pe alt branch — pentru lucru paralel fără conflicte.
Plan mode	Mod în care Claude planifică fără să modifice; aprobi înainte de execuție.
Effort level	Cât „gândește” înainte să acționeze (low→max). Pe Opus 4.8 default = high.
Checkpoint	Snapshot al fișierelor la fiecare mesaj; restaurabil cu <code>/rewind</code> .
Headless	Rulare non-interactivă (<code>claude -p</code>) pentru scripturi/CI.

27

Flux de lucru recomandat (rezumat)

1 Pregătește Convenții în <code>CLAUDE.md</code> . Task complex → <code>Shift+Tab</code> Plan mode, vezi planul.	2 Lasă-l să lucreze Accept edits; urmărești diff-urile inline. Urci <code>/effort</code> dacă e greu.	3 Verifică <code>! git diff</code> + <code>/code-review</code> + testele / <code>/verify</code> .	4 Închide curat <code>/clear</code> la schimbarea de task; <code>#</code> ce-a fost util.
---	--	--	--